A Deep Dive into Rainbow DQN

Chapter 4: Multi-step Learning

Harsh Verma

June 26, 2025

Abstract

Standard Q-learning updates are myopic, considering only the immediate reward and the estimated value of the very next state. This chapter explores Multi-step Learning, a powerful technique that bridges the gap between one-step Temporal-Difference (TD) learning and Monte Carlo (MC) methods. By incorporating a sequence of real rewards into the learning target, multi-step updates provide a more efficient mechanism for credit assignment and offer a crucial tool for managing the bias-variance trade-off, leading to faster and more stable learning.

1 The Spectrum of Learning Targets

To understand the "why" of multi-step learning, we must first understand the two extremes it interpolates between. This spectrum represents a fundamental trade-off in reinforcement learning between bias and variance.

1.1 One-step TD Learning (The Myopic Estimator)

This is the default approach in DQN. The learning target is formed by looking just **one step** into the future:

$$y_t^{(1)} = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-)$$

- **Low Variance:** The target's randomness comes only from a single transition (r_{t+1}, s_{t+1}). This makes the learning signal stable and less noisy, which is beneficial for the convergence of function approximators like neural networks.
- **High Bias:** The target is heavily "bootstrapped" from the value estimate $Q(s_{t+1}, a')$, which is itself an approximation. Any error in this estimate directly biases the target. This also leads to slow propagation of reward information, as credit for a reward received many steps in the future must be passed back one step at a time through the Bellman updates. This is particularly problematic in environments with sparse rewards.

1.2 Monte Carlo (MC) Learning (The Unbiased Observer)

At the other extreme, MC methods wait until the end of an episode and use the full, observed, discounted return G_t as the target.

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$$

- **Zero Bias:** The target is an empirical sample of the true value and contains no bootstrapped estimates. It is an unbiased estimate of $Q^\pi(s_t, a_t)$ under the policy π that was followed.
- **High Variance:** The return G_t is the sum of many random events (rewards and state transitions from the policy and environment). The same starting state-action can lead to vastly different returns, making the learning signal very noisy and potentially destabilizing training. Furthermore, updates can only happen at the end of an episode, which is inefficient.

The central challenge is clear: TD learning is stable but biased and slow; MC learning is unbiased but high-variance and often impractical for deep RL.

2 The N-step Solution: An Elegant Compromise

Multi-step learning offers a powerful solution by constructing a target that looks n steps into the future. It combines n steps of real, observed rewards with a bootstrapped value of the state encountered n steps away.

2.1 The N-step Target

The final n-step target combines the n-step return with a bootstrapped value from the state s_{t+n}:

$$y_t^{(n)} = \underbrace{\left(\sum_{k=0}^{n-1} \gamma^k r_{t+k+1}\right)}_{\text{n-steps of real rewards}} + \underbrace{\gamma^n \max_{a'} Q(s_{t+n}, a'; \theta^-)}_{\text{Bootstrapped value from n-steps away}} \tag{1}$$

If the episode terminates before n steps have passed (at time T), the sum is taken to the end, and the bootstrapped term becomes zero, effectively making it a Monte Carlo update for the remainder of the trajectory.

2.2 Managing the Bias-Variance Trade-off

The parameter n acts as a crucial knob for controlling bias and variance. By choosing a moderate value for n (e.g., $n = 3$ is common in Rainbow), we can achieve a "sweet spot" that significantly accelerates learning by allowing for faster credit assignment without introducing excessive variance. This is one of the most impactful yet computationally cheap improvements in Rainbow.

3 Implementation Details

3.1 The N-Step Buffer

Implementing multi-step learning requires a modification to the replay buffer logic. The most common approach is to use a small temporary buffer (a 'deque' of size n) to store the most recent transitions.

1. A new transition $(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1})$ is added to this deque.
2. If the deque has fewer than n items, we do nothing and wait for more transitions.
3. Once the deque is full, we can compute the n-step return for the oldest transition in it, $(s_{t-n+1}, a_{t-n+1}, \dots)$. The n-step experience, consisting of $(s_{t-n+1}, a_{t-n+1}, R^{(n)}_{t-n+1}, s_{t+1}, d_{t+1})$, is then added to the main replay buffer.

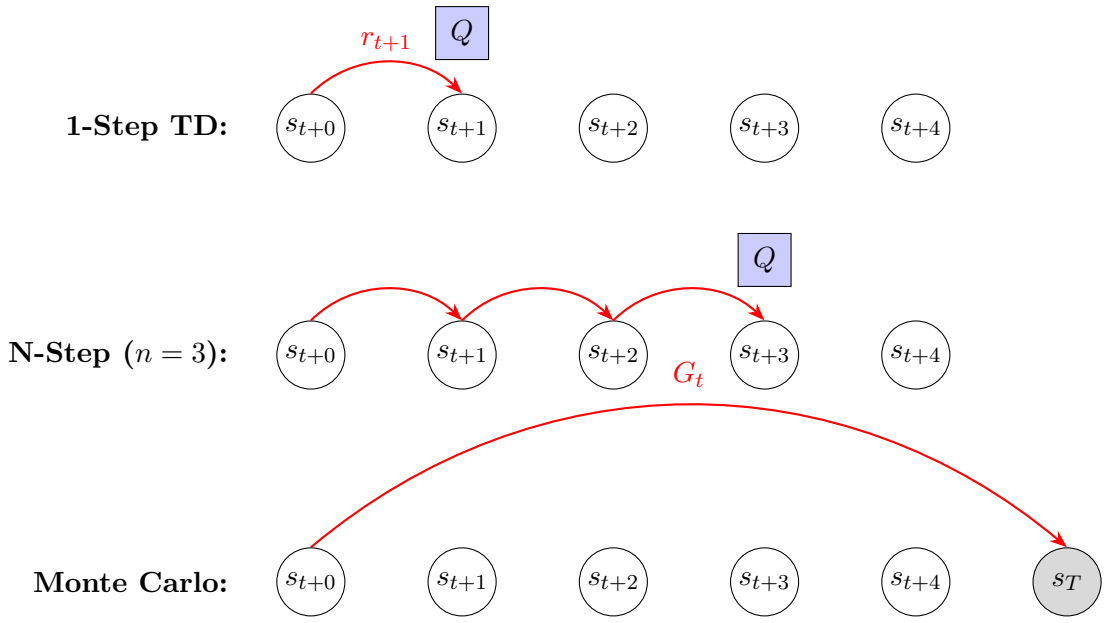

Figure 1: Backup diagrams illustrating the spectrum of learning updates. Red arrows indicate real rewards included in the target, while blue rectangles indicate where a bootstrapped value estimate is used.

This ensures that the main buffer stores complete n-step transitions ready for sampling. Special care must be taken to flush the temporary buffer at the end of an episode to create shorter, valid returns.

3.2 PyTorch Code Snippet

The following shows the core logic for calculating the n-step return and using it in the target computation.

```
# Assume a batch of n-step experiences has been sampled
# states, actions, n_step_rewards, final_next_states, final_dones = batch

# ... convert batch to tensors on the correct device ...

with torch.no_grad():
    # Use DDQN to get the value of the state n steps away
    online_q_next = online_net(final_next_states)
    best_actions = online_q_next.argmax(dim=1, keepdim=True)
    target_q_next = target_net(final_next_states).gather(1, best_actions)

    # Calculate the n-step target.
    # Note the discount factor is raised to the power of n for the bootstrapped value.
    y_target = n_step_rewards + (1 - final_dones) * (gamma ** n_steps) * target_q_next
```

Listing 1: Calculating the N-Step target value.

4 Discussion and Advanced Considerations

4.1 Off-Policy Learning

While highly effective, using large values of n in an off-policy setting (with a replay buffer) can be problematic. The actions in the n-step trajectory were taken by an older policy, but the

bootstrapped value $\max_{a'} Q(s_{t+n}, \dots)$ uses the *current* greedy policy. This mismatch can be an issue. However, for Q-learning, the 'max' operator makes it somewhat robust, which is why it works well in practice for moderate n. For on-policy methods like SARSA, this would require more complex corrections (e.g., importance sampling).

4.2 Synergy with Other Rainbow Components

Multi-step learning is a powerful enabler for other components:

- **Synergy with PER:** The learning signal from an n-step return is often more potent and less biased. When a significant reward is obtained, the n-step target will immediately reflect this, creating a large and meaningful TD-error. This provides a high-quality signal for PER to prioritize, allowing the agent to focus its attention on these causally-linked, long-horizon events.
- **Synergy with Distributional RL:** Instead of just shifting the mean value, the n-step return is used to shift the *entire* target distribution: $\mathcal{T}z_j = R^{(n)} + \gamma^n z_j$. This propagates rich distributional information much more quickly through the state space.